



Workflow: SwayLayoutManager

Laboratorio de Investigación y Desarrollo de
Software Libre (LIDSoL)

Areas:

Frontend

Backend

Date: December 17, 2025

Workflow Phases

1 Phase 1: Initial research and Planning

This phase will be a phase to focus on research for tools that can help for the development of the SwayLayoutManager project, also we will be planning the design and architecture of the project, by this I mean the actual code implementation structure, the modules that will compose the project, and how they will interact with each other.

1.1 Objectives

- Research existing tools and libraries that can facilitate interaction with Sway and Wayland.
- Propose a design and architecture for the SwayLayoutManager project.
- Create a plan with expected deadlines for the next phases of the project.

This phase won't last long since the heavy part for the command implementation is already defined in the proposal document. The main goal is to see if there is a more accurate way to implement commands rather than interact with `swaymsg` so we can improve performance and reliability.

As mentioned in the proposal document, the programming language selected for the project CLI is Go, this can be changed if we find a better alternative during this research phase.

2 Phase 2: Development for parsing tools

This phase will focus on the actual development of the SwayLayoutManager tool, specifically on the parsing of the JSON presets that will be used to save layouts. This phase will be the first code implementation phase, where we will start writing the code that will form the basis of the project, taking into consideration the programming language and libraries selected in phase 1.

2.1 Save Command Implementation

The `swaylayoutmgr save <name>` command is the core functionality that captures the current state of all active workspaces and saves it as a named layout preset. This command will be implemented with the following detailed specification:

2.1.1 Command Execution Flow

1. **Workspace Discovery:** Execute `swaymsg -t get_tree | jq`(to be determined) to obtain the current layout tree and identify all active workspaces along with their contained windows.
2. **Data Parsing:** Parse the JSON output to extract relevant information for each window in the specified workspaces, including:
 - Application identifier (`app_id` for Wayland apps, `class` for X11 apps)
 - Window title (`name` field) for additional matching criteria
 - Workspace number/name (`workspace`)
 - Container layout type (`layout`: `splith`, `splitv`, `tabbed`, `stacked`)
 - Relative position within parent container (`rect` coordinates and sibling order)
 - Workspace representation
 - Window dimensions and floating status (`floating`, `rect`)
 - Output/monitor association
3. **Preset Storage:** Save the structured data as a JSON file in `~/.config/swaylayoutmanager/presets/` with metadata including creation timestamp and sway version compatibility.
4. **Validation:** Verify the preset file is valid JSON and contains all necessary fields before confirming successful save operation with log output.

2.1.2 Error Handling Requirements

- Check if preset name already exists and prompt for overwrite confirmation
- Validate that sway IPC socket is accessible (to be determined)
- Handle cases where some applications don't have reliable identifiers (to be determined)
- Create configuration directory structure if it doesn't exist
- Provide meaningful error messages for troubleshooting

2.1.3 JSON Structure Design

The saved preset file will contain structured data that enables accurate restoration of workspace layouts, including all necessary metadata for future compatibility and validation.

3 Phase 3: Development for Layout Restoration

This phase will focus on implementing the layout restoration functionality of the SwayLayoutManager tool. By building with the parsing and saving capabilities developed in phase 2, this phase will implement the `load` command that reads saved presets and reconstructs workspace layouts by launching applications and positioning them according to the saved configuration.

3.1 Load Command Implementation

The `swaylayoutmgr load <name>` command is the complementary functionality to the `save` command, enabling users to restore previously saved workspace layouts. This command will be implemented with the following detailed specification:

3.1.1 Command Execution Flow

1. **Preset Loading:** Read the configuration file from `~/.config/swaylayoutmanager/presets/<name>.json` and parse the JSON structure to extract the saved layout information.
2. **Workspace Restoration:** Iterate over each workspace defined in the preset and for each application window:
 - Switch to the target workspace using `swaymsg workspace number <N>`
 - Check if the application is already running by querying `app_id/class`
 - Launch the application using `swaymsg exec <command>` if not already running
 - Wait for the window to appear (configurable timeout)
 - Use sway's criteria selection `[app_id="..." title="..."]` to focus the specific window
 - Apply positioning commands to move and resize the window according to saved coordinates
 - Restore floating state if applicable (`swaymsg floating enable/disable`)
 - Set the workspace layout mode (`splith`, `splitv`, `tabbed`, or `stacked`)
3. **Layout Reconstruction:** Apply the saved layout structure by recreating container hierarchies and positioning windows according to their saved `rect` coordinates and sibling order.
4. **Verification:** Query the current layout tree to verify successful restoration and log any discrepancies or applications that failed to launch.

3.1.2 Error Handling Requirements

- Check if preset file exists before attempting to load
- Validate JSON structure and presence of all required fields
- Handle missing applications gracefully by logging warnings and continuing with remaining applications
- Implement timeout handling for applications that fail to launch or don't create windows within expected time

- Provide fallback positioning strategies if exact layout reconstruction fails (to be determined)
- Handle version compatibility issues with presets created using different sway versions (to be determined)
- Provide detailed error messages and restoration reports for troubleshooting

3.1.3 Implementation Considerations

The load command will need to handle asynchronous application launches and implement proper synchronization mechanisms to ensure windows are fully initialized before applying layout commands.

4 Phase 4: Enhanced Functionality and Flag System

This phase focuses on refining the layout restoration implementation from Phase 3 and introducing a flexible flag system for workspace operations. The improvements address real-world usage patterns and provide users with fine-grained control over save and load operations.

4.1 Load Command Implementation Updates

Starting from Phase 3 documentation, the load command has been refined with the following improvements and clarifications:

4.1.1 New Implementation Approach

The final implementation uses a **direct restoration approach** that differs from the initial specification:

- **No Explicit Coordinate Positioning:** Instead of applying move/resize commands to match saved coordinates, this implementation uses Sway’s automatic tiling system combined with container hierarchy reconstruction.
- **Sequential Launching:** Applications are launched sequentially into target workspaces, with split commands applied between siblings to maintain proper layout structure.

4.1.2 Direct Container Restoration

The `restoreContainerDirect()` method implements the core restoration logic:

1. For single windows with `ExecCommand`, launch the application directly using `exec <command>`
2. For grouped containers (tabbed/stacked):
 - Launch the first window
 - Set the container layout mode (`layout tabbed` or `layout stacked`)
 - Launch remaining windows, which automatically join the group
3. For split containers:
 - Process children sequentially
 - Apply `split h/v` commands between siblings
 - Recurse into child containers

4.1.3 Enhanced Synchronization

To ensure proper window initialization and layout application:

- **Deadline-based Timeouts:** `waitForContainerWindows()` uses a deadline approach (default 5 seconds) to wait for all windows in a container to appear
- **Strategic Delays:** Brief delays between operations (100-400ms) allow windows to stabilize:
 - 100ms after workspace switch and split commands

- 300ms between sibling containers
- 400ms after application launch
- **Leaf Window Collection:** `collectLeafWindows()` traverses the container tree to identify all windows requiring launch, enabling proper wait synchronization

4.1.4 Restoration Reporting

The `RestoreResult` struct provides comprehensive restoration feedback:

- **WorkspacesRestored:** Count of successfully restored workspaces
- **ApplicationsLaunched:** Number of applications started
- **ApplicationsFailed:** Count of applications that failed to launch
- **Errors:** Detailed error list for troubleshooting

Output example:

Layout 'my-workspace' restoration complete:

```
- Workspaces restored: 3
- Applications launched: 8
- Applications failed: 1
```

Warnings/Errors encountered:

```
- Failed to launch: invalid-command
```

4.2 Flag System Implementation

Phase 4 introduces a flexible flag parsing system that enables control over all command operations. This system is designed for extensibility, allowing multiple flags to be combined for complex behaviors.

4.2.1 Available Flags

The following flags provide comprehensive control over save and load operations:

Workspace Filtering Flags

- `--skip-workspace=<id>[,<id>...]`
Excludes specified workspace(s) from save or restore operations. Accepts workspace numbers or names (comma-separated for multiple workspaces).
 - *Applicable to:* `save`, `load`
 - *Example:* `swaylayoutmgr save my-layout --skip-workspace=1,2,scratch`
 - *Use case:* Exclude temporary workspaces from layouts
- `--only-workspace=<id>[,<id>...]`
Includes only the specified workspace(s) in save or restore operations. All other workspaces are ignored.

- *Applicable to:* `save`, `load`
- *Example:* `swaylayoutmgr save coding-setup --only-workspace=2,3`
- *Use case:* Create focused layouts containing only relevant workspaces
- *Note:* Cannot be combined with `--skip-workspace`

Application Filtering Flags

- `--skip-app=<id>[,<id>...]`
Excludes specified application(s) by `app_id` or window class name. Supports comma-separated lists.
 - *Applicable to:* `save`, `load`
 - *Example:* `swaylayoutmgr save layout --skip-app=discord,Spotify`
 - *Use case:* Exclude distracting applications
- `--only-app=<id>[,<id>...]`
Includes only specified application(s) in the layout. All other applications are filtered out.
 - *Applicable to:* `save`, `load`
 - *Example:* `swaylayoutmgr save browsers --only-app=firefox,chromium`
 - *Use case:* Create application-specific layouts (e.g., "terminal-only")
 - *Note:* Cannot be combined with `--skip-app`

Window Type Filtering

- `--ignore-floating`
Excludes all floating windows from save or restore operations. Only tiled, tabbed, and stacked windows are included.
 - *Applicable to:* `save`, `load`
 - *Example:* `swaylayoutmgr save clean-layout --ignore-floating`
 - *Use case:* Create layout presets without floating windows

Restoration Behavior Flags

- `--clear`
Closes all windows in workspaces that will be restored before applying the layout.
 - *Applicable to:* `load`
 - *Example:* `swaylayoutmgr load my-layout --clear`
 - *Use case:* Ensure clean workspace state before restoration
 - *Behavior:* Only clears workspaces present in the saved layout

- **--clear-all**
Closes all windows in every workspace before restoring the layout.
 - *Applicable to:* load
 - *Example:* `swaylayoutmgr load fresh-start --clear-all`
 - *Use case:* Complete session reset with fresh layout
 - *Warning:* Closes windows in all workspaces, not just those being restored
- **--reuse-all**
Attempts to reuse existing windows instead of launching new application instances for all applications in the layout.
 - *Applicable to:* load
 - *Example:* `swaylayoutmgr load layout --reuse-all`
 - *Use case:* Reorganize existing windows without spawning duplicates
 - *Behavior:* Matches windows by `app_id` or window class, moves them to target workspaces
- **--reuse=<app_id>[,<app_id>...]**
Attempts to reuse existing windows only for specified applications. Other applications launch new instances.
 - *Applicable to:* load
 - *Example:* `swaylayoutmgr load layout --reuse=firefox,thunderbird`
 - *Use case:* Reuse single-instance applications while launching fresh terminals/editors
 - *Note:* Cannot be combined with `--reuse-all`

Focus Control Flags

- **--focus-workspace=<id>**
After restoration completes, switches focus to the specified workspace by number or name.
 - *Applicable to:* load
 - *Example:* `swaylayoutmgr load layout --focus-workspace=1`
 - *Use case:* Ensure consistent workspace context after layout restoration
 - *Note:* Cannot be combined with `--focus-app`
- **--focus-app=<id>**
After restoration completes, focuses the specified application by `app_id` or window class.
 - *Applicable to:* load
 - *Example:* `swaylayoutmgr load layout --focus-app=firefox`
 - *Use case:* Direct attention to primary application after restoration
 - *Behavior:* If multiple instances exist, uses Sway’s default focus behavior

- *Note:* Cannot be combined with `--focus-workspace`

File Operation Flags

- `--export=<path>`
Saves the layout to a specified file path instead of the default presets directory.
 - *Applicable to:* `save`
 - *Example:* `swaylayoutmgr save --export=/tmp/backup-layout.json`
 - *Use case:* Create layout files in custom locations for backup or sharing
- `--import=<path>`
Loads a layout from a specified file path instead of the default presets directory.
 - *Applicable to:* `load`
 - *Example:* `swaylayoutmgr load --import=/tmp/backup-layout.json`
 - *Use case:* Restore layouts from custom locations or shared files
- `--overwrite`
When saving, overwrites an existing preset with the same name without prompting for confirmation.
 - *Applicable to:* `save`
 - *Example:* `swaylayoutmgr save my-layout --overwrite`
 - *Use case:* Automated scripts, updates to existing layouts
 - *Default:* Without this flag, users have to delete existing presets manually before saving with the same name

4.2.2 Flag Usage Examples

Example 1: Focused Development Layout Save only workspaces 2 and 3, excluding communication apps:

```
swaylayoutmgr save dev-focus \  
--only-workspace=2,3 \  
--skip-app=discord,slack,thunderbird \  
--ignore-floating
```

Example 2: Clean Restoration Restore layout with all windows cleared and reuse browser instances:

```
swaylayoutmgr load my-layout \  
--clear-all \  
--reuse=firefox,chromium \  
--focus-workspace=1
```

Example 3: Portable Layout Export Export a browser-only layout to a shareable file:

```
swaylayoutmgr save \  
  --only-app=firefox,chromium \  
  --export=~/.layouts/browsers.json \  
  --ignore-floating
```

Example 4: Selective Workspace Clear Restore specific workspaces with clean slate:

```
swaylayoutmgr load work-setup \  
  --only-workspace=1,2,3 \  
  --clear \  
  --focus-workspace=1
```

Example 5: Exclude Temporary Workspaces Save layout excluding scratch and temporary workspaces, reuse all existing windows, and focus a specific application:

```
swaylayoutmgr save main-layout \  
  --skip-workspace=scratch,temp,9 \  
  --overwrite  
swaylayoutmgr load main-layout \  
  --reuse-all \  
  --focus-app=code
```

Example 6: Import and Update Layout Import a layout from a custom location and update the existing preset:

```
swaylayoutmgr load \  
  --import=~/.shared-layouts/team-setup.json \  
  --clear \  
  --focus-app=firefox  
swaylayoutmgr save team-setup \  
  --skip-app=spotify \  
  --overwrite
```

5 Phase 5: Sway Layout Manager Configuration System

This phase focuses on the development of a centralized configuration system for SwayLayoutManager. By implementing program specific settings, this phase enables user-defined defaults, streamlining the use of the CLI tool and reducing the need to repeatedly provide complex flag combinations for standard operations.

5.1 Objectives

- Implement a global configuration file (e.g., YAML, TOML, or JSON) to manage persistent user preferences.
- Define customizable default behaviors for **save** and **load** operations to minimize repetitive flag usage.
- Establish user-adjustable parameters for restoration synchronization, such as strategic delays and timeouts.
- Ensure a clear hierarchy of configuration precedence, where command-line flags override configuration file defaults.

5.2 Configuration File Implementation

The tool will introduce support for a user-level configuration file, typically located at `~/.config/sway-layout-`. This file will govern the baseline behavior of the command-line interface.

5.2.1 Configuration Schema

The configuration file will be structured into several logical sections:

- **General Settings:** Definitions for default directories, such as custom paths for saving, loading, and exporting presets if they differ from the default **presets** folder, also it will include options for preset save and restoration behavior, based on the flags defined in Phase 4.
- **Synchronization Timing:** User-adjustable values for the strategic delays utilized in the restoration logic. This allows users on varying hardware to tweak the 100ms–400ms stabilization windows for optimal performance and layout reliability.
- **Preset Preferences:** Default layout exclusions or inclusions, enabling users to permanently ignore certain scratchpad or temporary workspaces without needing to type the `--skip-workspace` flag on every run.
- **Preset:** Options for a specific preset, this will be a section that allows user to define settings for specific preset(s), this section(s) will override general settings for the preset(s) defined in it, this will allow users to have different default behaviors for different presets.

5.2.2 Configuration Hierarchy

1. **Command-Line Flags:** Explicit flags passed during execution (e.g., `--clear`) will have a higher precedence than any settings defined in the configuration file, ensuring that users can

override defaults on a per-command basis when necessary.

2. **Configuration File:** Settings defined in the `~/.config/sway-layout-manager/config.yaml` file will act as the secondary source of truth.
3. **Hardcoded Defaults:** If a setting is absent from both the CLI arguments and the configuration file, the application will fall back to its internal defaults.

5.3 Implementation Considerations

- **Backward Compatibility:** The introduction of the configuration system must not break the parsing and restoration of JSON presets created during Phases 2 and 3.
- **Validation and Error Handling:** The CLI must rigorously validate the configuration file schema upon startup. If the file contains invalid syntax or unsupported fields, the tool should provide meaningful error messages indicating the exact line and nature of the configuration error before halting execution.
- **Default Generation:** If no configuration file is found, the tool should operate seamlessly on internal defaults, and optionally provide a command (e.g., `swaylayoutmgr init`) to generate a boilerplate configuration file for the user.